

## File Handling

File handling in python is a way to work with files using python programs. It helps us to create files, read data from files, write data into files, update files, and delete files using a python program.

Normally, when a program runs, data is stored in memory (RAM). Once the program stops, the data is lost. File handling allows us to store data permanently in files so that it can be accessed later.

### Why Do We Need File Handling?

Without file handling:

- Data exists only while the program is running.
- Data is lost when the program closes.

With file handling:

- Data is permanently stored.
- Data can be shared between applications.
- Large amounts of information can be managed efficiently.

Example:

```
file = open("student.txt", "w")
file.write("priya")
file.close()
```

output: Priya

### File:

A file is a collection of related information stored on a storage device such as a hard disk, SSD, or USB drive. Files are used to save data permanently so that it can be accessed and modified whenever required.

Examples of files include text documents, images, videos, PDFs, spreadsheets, and log files.

Examples:

- student.txt
- employees.csv
- report.pdf
- image.jpg

Example:

```
file = open("employee.txt", "w")
file.write("employee data")
file.close()
```

output: A file named employee.txt is created.

### Creating a File:

Creating a file in Python means generating a new file on the computer using a Python program. Files are created to store information permanently so that it can be used later.

Python provides different modes such as w and x to create files.

Example:

```
file = open("student.txt", "w")  
file.close()
```

output: After execution, a new file named student.txt is created.

## Text File:

A text file is a type of file that stores data in a human-readable format. The contents of a text file can be viewed and edited using text editors such as Notepad, VS Code, or Sublime Text.

Text files store information as characters and are commonly used for storing notes, reports, configuration data, and application logs.

Examples

- .txt files
- .csv files
- .json files
- .xml files

```
file = open("notes.txt", "w")  
file.write("python notes")  
file.close()
```

output:python notes

## Binary File:

A binary file is a file that stores data in binary format (0s and 1s) instead of readable text. Binary files are generally used to store images, videos, audio files, and documents.

Unlike text files, binary files cannot be easily understood by humans because their contents are stored in machine-readable format.

Examples

- image.jpg
- video.mp4
- music.mp3
- document.pdf

```
file = open("image.bin", "wb")  
file.write(b"hello")  
file.close()
```

Reading Binary File:

```
file = open("image.bin", "rb")  
print(file.read())  
file.close()
```

output:b"hello"

## open() Function:

The open() function is used to open an existing file or create a new file for performing operations such as reading, writing, or appending data.

Before performing any operation on a file, Python must establish a connection between the program and the file. This connection is created using the `open()` function.

**Syntax:**

`file = open("filename", "mode")`

**Parameters:**

`filename` → Name of the file to be opened.

`mode` → Specifies the operation to be performed on the file.

Example:

```
file = open("student.txt", "r")
print("file opened successfully")
file.close()
```

output:file opened successfully

In this example:

- student.txt is the file name.
- r represents read mode.

**File Modes:**

File modes specify how a file should be opened and what operations are allowed on that file. Different modes are available for reading, writing, appending, and creating files.

**Common File Modes:**

| Mode | Meaning                 |
|------|-------------------------|
| "r"  | Read file               |
| "w"  | Write file              |
| "a"  | Append data             |
| "x"  | Create new file         |
| "r+" | Read and write          |
| "rb" | Binary file for reading |
| "rw" | Binary file for write   |
| "a+" | Appending and reading   |

**Read Mode (r):**

Read mode is used to open a file for reading its contents. In this mode, data can only be viewed and cannot be modified.

The file must already exist; otherwise, Python raises a `FileNotFoundError`.

Use Cases

- Reading reports
- Reading configuration files
- Viewing stored records

Example: used to read existing files

```
file = open("student.txt", "r")
print(file.read())
file.close()
```

output: Python  
Django  
Flask

### **Write Mode (w):**

Write mode is used to write data into a file. If the file already exists, all existing content is removed and replaced with new content. If the file does not exist, Python creates a new file automatically.

Use Cases

- Creating reports
- Saving user information
- Exporting data

Example:

```
file = open("student.txt", "w")  
file.write("Django")  
file.close()
```

output: Django

### **Append Mode (a):**

Append mode is used to add new data at the end of an existing file without removing the previous content.

This mode is useful when historical records need to be maintained.

Use Cases

- Attendance systems
- Transaction records
- Application logs

Example:

```
file = open("student.txt", "a")  
file.write("\nFastAPI")  
file.close()
```

output:FastAPI

### **Reading a File:**

Reading a file means retrieving the information stored inside it. Python provides different methods to read data from files depending on the requirement.

Benefits

- Access stored data
- Generate reports
- Analyze information

Example:

```
file = open("student.txt", "r")  
data = file.read()  
print(data)  
file.close()
```

output:Django

## Writing to a File:

Writing to a file means storing new information inside a file. This operation is commonly used to save user data, create reports, and generate output files.

- Benefits
- Permanent storage
- Easy data management
- Information backup

Example:

```
file = open("marks.txt", "w")
file.write("total marks:95")
file.close()
```

output: total marks:95

## File Pointer:

A file pointer is an internal cursor that keeps track of the current position within a file. Whenever data is read or written, the file pointer automatically moves forward.

The file pointer helps Python know where the next read or write operation should take place.

Example:

```
file = open("student.txt", "r")
print(file.read(3))
print(file.tell())
file.close()
```

output:pyt

3

## seek() Method:

The seek() method is used to move the file pointer to a specific location within the file.

It allows random access to data rather than reading the file from the beginning every time.

Benefits

- Faster access to data
- Random file navigation
- Efficient processing of large files

Example:

```
file = open("student.txt", "r")
file.seek(2)
print(file.read())
file.close()
```

if file contains : python

output: thon

## tell() Method:

The tell() method returns the current position of the file pointer within the file.

It is useful when tracking the location from which data is being read or written.

#### Benefits

- Position tracking
- Debugging file operations
- Better file management

#### Example:

```
file = open("student.txt", "r")
print(file.tell())
file.read(4)
print(file.tell())
file.close()
```

output: 0

4

#### **close() Method:**

The close() method is used to close an opened file after all operations have been completed.

Closing a file releases system resources and ensures that all data is saved properly.

#### Importance

- Prevents memory leaks
- Improves performance
- Protects data integrity

#### **with Statement:**

The with statement is the recommended way of working with files in Python. It automatically closes the file after operations are completed, even if an error occurs during execution.

#### Benefits

- Automatic file closing
- Cleaner code
- Better exception handling
- Improved resource management

#### Example:

```
with open("student.txt", "r") as file:
    print(file.read())
```

output: Python

Django

File is automatically closed

#### **readline() Method:**

The readline() method is used to read **one line at a time** from a file. Unlike read(), which reads the entire file, readline() reads only a single line and moves the file pointer to the next line.

## Why Do We Use readline()?

- To read files line by line
- To process large files efficiently
- To save memory
- To access specific lines

Example File:

Assume student.txt contains:

```
Priya
Rahul
Anjali
kiran
```

Reading First Line:

```
file = open("student.txt", "r")
print(file.readline())
file.close()
```

output: Priya

Reading Multiple Lines:

```
file = open("student.txt", "r")
print(file.readline())
print(file.readline())
print(file.readline())
file.close()
```

Output: Priya

Rahul

Anjali

## Deleting a File in Python:

Deleting a file in Python means removing a file permanently from your computer using a Python program. After a file is deleted, it cannot be used for reading, writing, or updating.

Python provides the `os.remove()` function to delete files.

## Why Do We Need to Delete Files?

Sometimes files are no longer needed and occupy unnecessary storage space. Deleting such files helps keep the system organized and improves storage management.

For example:

- Removing old reports
- Deleting temporary files
- Removing outdated records
- Cleaning log files

## How Python Deletes a File

To delete a file, Python uses the `os` module.

### Step 1: Import the os Module

```
import os
```

## Step 2: Delete the File

```
os.remove("student.txt")
```

This statement deletes the file named student.txt.

Example:

```
import os
os.remove("student.txt")
```

The file will be removed from your system.

## Checking Before Deleting

If the file does not exist, Python will show an error.

To avoid this, first check whether the file exists.

```
import os
if os.path.exists("student.txt"):
    os.remove("student.txt")
    print("File deleted successfully")
else:
    print("File not found")
```

output: File deleted successfully  
or File not found

## Advantages of Deleting Files

- Frees storage space
- Removes unwanted files
- Keeps folders organized
- Improves file management

## Important Points

- Deleted files are removed permanently.
- Always check if the file exists before deleting.
- Use exception handling for safer programs.
- Be careful while deleting important files.

## Exception Handling in File Operations:

Exception handling is used to manage errors that may occur while working with files. It prevents the program from crashing and allows developers to handle errors gracefully.

Example:

```
try:
    file = open("data.txt", "r")
    print(file.read())
except FileNotFoundError:
    print("File not found")
```

output: File not found

## Benefits

- Improved reliability
- Better user experience
- Safer file operations